

Rapport Hebdomadaire B2-CP3 : Base de Données et Sécurité

Projet : Mini CRM Auteur : Omer Atici Date : Avril 2026

1. Modélisation de la Base de Données (MCD → MLD → MPD)

Pour mon projet de CRM, j'ai conçu le système de base de données de manière à gérer efficacement mes clients et mes devis.

1.1 Mon Modèle Conceptuel des Données (MCD)

Dans ma conception :

- Un **Utilisateur** gère plusieurs **Clients** et émet plusieurs **Devis**.
- Un **Client** est géré par un seul **Utilisateur** et peut recevoir plusieurs **Devis**.
- Un **Devis** appartient à un seul **Client**, est créé par un seul **Utilisateur**, et contient plusieurs **Lignes de Devis**.

1.2 Mon Modèle Logique des Données (MLD)

J'ai décliné ces entités en plusieurs tables logiques pour mon application :

- **User** (id, email, password, nom, prenom, createdAt)
- **Client** (id, nom, prenom, email, telephone, entreprise, adresse, createdAt, updatedAt, *userId*)
- **Devis** (id, numero, statut, dateEmission, dateEcheance, totalHT, totalTTC, tva, notes, createdAt, updatedAt, *userId*, *clientId*)
- **LigneDevis** (id, description, quantite, prixUnitaire, total, *devisId*)

1.3 Normalisation et Optimisation de ma base

J'ai veillé à ce que ma base de données respecte scrupuleusement la **3e Forme Normale (3NF)** :

- Je n'ai aucune redondance : un client ou un collaborateur n'est jamais saisi deux fois.
- J'utilise strictement des clés étrangères (*userId* , *clientId*) pour maintenir l'intégrité de mes données.
- J'ai implémenté une suppression en cascade (**ON DELETE CASCADE**) sur mes Lignes de Devis : si je supprime un devis, les lignes associées disparaissent proprement sans polluer ma base avec des données fantômes.

2. Mes Scripts de Création et Adaptation (SQL)

J'ai fait le choix d'utiliser **PostgreSQL** couplé à l'ORM **Prisma**. Voici la structure SQL exacte (MPD) que j'ai implémentée pour mon serveur :

```
CREATE TABLE "User" (  
  "id" SERIAL PRIMARY KEY,  
  "email" VARCHAR(255) UNIQUE NOT NULL,  
  "password" VARCHAR(255) NOT NULL,  
  "nom" VARCHAR(255) NOT NULL,  
  "prenom" VARCHAR(255) NOT NULL,
```

```

        "createdAt" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
    );

CREATE TABLE "Client" (
    "id" SERIAL PRIMARY KEY,
    "nom" VARCHAR(255) NOT NULL,
    "prenom" VARCHAR(255) NOT NULL,
    "email" VARCHAR(255),
    "telephone" VARCHAR(255),
    "entreprise" VARCHAR(255),
    "userId" INTEGER NOT NULL REFERENCES "User"("id")
);

CREATE TABLE "Devis" (
    "id" SERIAL PRIMARY KEY,
    "numero" VARCHAR(255) UNIQUE NOT NULL,
    "statut" VARCHAR(255) NOT NULL DEFAULT 'brouillon',
    "totalTTC" DOUBLE PRECISION NOT NULL DEFAULT 0,
    "userId" INTEGER NOT NULL REFERENCES "User"("id"),
    "clientId" INTEGER NOT NULL REFERENCES "Client"("id")
);

CREATE TABLE "LigneDevis" (
    "id" SERIAL PRIMARY KEY,
    "description" VARCHAR(255) NOT NULL,
    "quantite" DOUBLE PRECISION NOT NULL,
    "prixUnitaire" DOUBLE PRECISION NOT NULL,
    "total" DOUBLE PRECISION NOT NULL,
    "devisId" INTEGER NOT NULL REFERENCES "Devis"("id") ON DELETE CASCADE
);

```

3. Mon Jeu de Données de Test (Seeding)

Pour valider et démontrer le fonctionnement de mon application, j'ai développé mon propre script de "seeding". Celui-ci génère automatiquement un jeu de données de test pertinent (des commerciaux collaborateurs, une liste de clients et des devis naviguant entre plusieurs statuts).

Voici l'équivalent cognitif SQL de ce que mon script déclenche en coulisses lorsque je lance ma commande d'initialisation `npm run prisma db seed` :

```

-- J'insère un Utilisateur / Commercial (Hash bcrypt généré via mon script Node.js)
INSERT INTO "User" (email, password, nom, prenom)
VALUES ('jean.dupont@minicrm.fr', '$2a$10$...', 'Dupont', 'Jean');

-- J'insère un de ses Clients
INSERT INTO "Client" (nom, prenom, email, adresse, "userId")
VALUES ('Martin', 'Paul', 'pmartin@entreprise-a.fr', '10 Rue de Paris, 75001', 1);

-- J'insère un Devis pour ce client
INSERT INTO "Devis" (numero, statut, "totalHT", "totalTTC", "userId", "clientId")
VALUES ('DEV-2026-001', 'accepté', 1500, 1800, 1, 1);

```

```
-- J'injecte les Lignes de prestation de ce devis
INSERT INTO "LigneDevis" (description, quantite, "prixUnitaire", total, "devisId")
VALUES
('Création de la page vitrine', 1, 1000, 1000, 1),
('Hébergement annuel', 1, 500, 500, 1);
```

4. Sécurité, Confidentialité des données et RGPD

S'agissant d'un CRM qui manipule les informations personnelles de prospects, c'est un point critique de mon cahier des charges. Pour sécuriser mon application, j'ai mis en place les architectures suivantes :

4.1 Sécurité et Accès

- **Cryptographie des mots de passe** : Je hache tous les mots de passe de mes utilisateurs de façon asynchrone avec la librairie `bcrypt` . Je génère un "sel" (salt) unique de 10 tours pour empêcher les tentatives de décryptage par tables arc-en-ciel.
- **Sécurité des requêtes** : Mon choix d'utiliser l'ORM Prisma me garantit que toutes mes requêtes SQL sont nativement paramétrées, me protégeant des classiques injections SQL.
- **Authentification Robuste** : Mon API back-end distribue des sessions via des **JWT (JSON Web Token)** signés sur 24h. Je stocke temporairement ce token dans le `localStorage` de mon front-end, et je blinde mes routes back-end avec le middleware `authentication.middleware.js` qui exige le retour précis de ce token dans un en-tête `Authorization: Bearer` . J'ai parfaitement conscience de la théorie des failles XSS, c'est pourquoi je m'appuie sur le composant de mon framework front-end qui échappe nativement le code pour désinfecter toutes les entrées utilisateurs.

4.2 Ma conformité au RGPD

- **Cloisonnement Strict (Privacy by design)** : J'ai conçu la base de données de sorte qu'il soit physiquement impossible pour un commercial de lister les données clients d'un collègue. Ma clé d'appartenance relationnelle `userId` est croisée avec l'identité du JWT lors de chaque requête API.
- **Minimisation des données (Pertinence et Transparence)** : Je récolte uniquement les informations personnelles strictement justifiées par l'obligation légale de la production d'un devis comptable, assurant ma conformité au principe de minimisation du RGPD.